



LEARNING DYNAMICS OF LLM FINETUNING

LLM Systems
ICLR Best Paper 2025

X-LAB Best Paper Reading Series

Utkarsh Kumar

9/4/2026

Motivation::

When a neural network learns from one example, it doesn't just update its answer for *that* example — it nudges its answers for everything that looks similar:

1. **SFT (instruction tuning) ripples explain a kind of hallucination.** When you train the model on question A with answer A, the probability of *other* training answers (answer B, for a different question) also rises when asked question A. So, after SFT, the model may answer question A with pieces of answer B
2. **DPO has a "squeezing effect."** DPO pushes the rejected answer *down*. But softmax has a quirk: when you push down on something the model already thinks is very unlikely, the freed probability doesn't spread evenly

Danica J. Sutherland (she)

Associate Professor, [UBC Computer Science](#) [Canada CIFAR AI Chair](#), [Amii](#)

[UBC Machine Learning](#) [AML-TN](#) [MILD](#) (ML theory) [CAIDA](#) (AI)

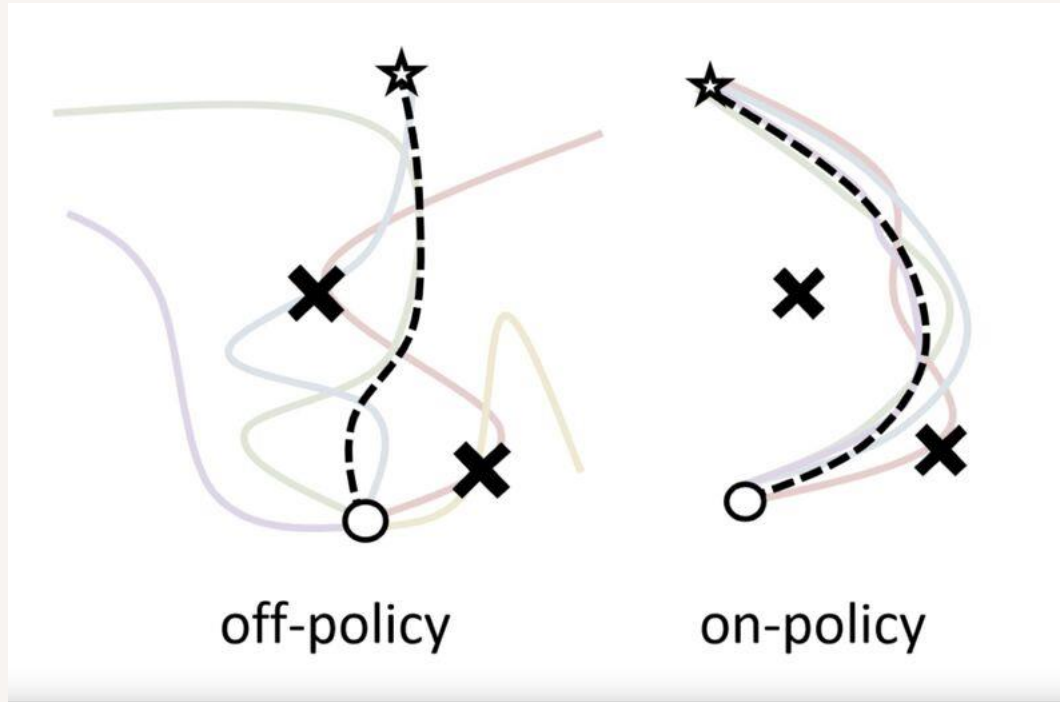
[PIHOT/Kantorovich Initiative](#) (optimal transport)

[Queer in AI](#) [Name Change Policy Working Group](#)

[dsuth\[at\]cs.ubc.ca](mailto:dsuth[at]cs.ubc.ca) or [djs\[at\]djsutherland.ml](mailto:djs[at]djsutherland.ml)

[CV](#) [orcid](#) [github](#) [crossvalidated](#) [bsky](#)

Basics: Off Policy v On Policy



Defining Learning Dynamics::

$$\Delta\theta \triangleq \theta^{t+1} - \theta^t = -\eta \cdot \nabla \mathcal{L}(f_\theta(\mathbf{x}_u), \mathbf{y}_u), \quad \Delta f(\mathbf{x}_o) \triangleq f_{\theta^{t+1}}(\mathbf{x}_o) - f_{\theta^t}(\mathbf{x}_o)$$

u is the current training sample, o is the current inference

After a GD update on $\mathbf{x}_u$, how does the model's prediction on $\mathbf{x}_o$ change?

$$\mathbf{x}_o \xrightarrow{h_\theta} \underbrace{z}_{\text{logits}} = h_\theta(\mathbf{x}_o) \in \mathbb{R}^{V \times L} \quad \implies \quad \pi = \text{Softmax_column}(z)$$

$$\Delta \log \pi^t(\mathbf{y} \mid \mathbf{x}_o) \triangleq \log \pi_{\theta^{t+1}}(\mathbf{y} \mid \mathbf{x}_o) - \log \pi_{\theta^t}(\mathbf{y} \mid \mathbf{x}_o)$$

Defining Learning Dynamics::

After 1st order Taylor series expansion

$$\begin{aligned}
 \Delta \log \pi^t(\mathbf{y} \mid \mathbf{x}_o) &= \overbrace{\nabla_{\theta} \log \pi^t(\mathbf{y} \mid \mathbf{x}_o) \big|_{\theta_t}}^{V \times d} \overbrace{\Delta \theta}^{d \times 1} + O(\|\Delta \theta\|^2) \\
 &= \left[\overbrace{\nabla_z \log \pi^t(\mathbf{x}_o)}^{V \times V} \overbrace{\nabla_{\theta} z(\mathbf{x}_o)}^{V \times d} \right] \left[-\eta \overbrace{\nabla_{\theta} z(\mathbf{x}_u)^{\top}}^{d \times V} \overbrace{\nabla_z \mathcal{L}(\mathbf{x}_u)^{\top}}^{V \times 1} \right] \\
 &= -\eta \mathcal{A}^t(\mathbf{x}_o) \underbrace{\left[\nabla_{\theta} z(\mathbf{x}_o) \nabla_{\theta} z(\mathbf{x}_u)^{\top} \right]}_{= \mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u), \text{ the eNTK}} \mathcal{G}^t(\mathbf{x}_u, \mathbf{y}_u)
 \end{aligned}$$

$$\underbrace{\Delta \log \pi^t(\mathbf{y} \mid \mathbf{x}_o)}_{V \times 1} = -\eta \underbrace{\mathcal{A}^t(\mathbf{x}_o)}_{V \times V} \underbrace{\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u)}_{V \times V} \underbrace{\mathcal{G}^t(\mathbf{x}_u, \mathbf{y}_u)}_{V \times 1} + O(\eta^2 \|\nabla_{\theta} \mathbf{z}(\mathbf{x}_u)\|_{\text{op}}^2)$$

Interpretations of matrices::

$$\underbrace{\Delta \log \pi^t(\mathbf{y} \mid \mathbf{x}_o)}_{V \times 1} = -\eta \underbrace{\mathcal{A}^t(\mathbf{x}_o)}_{V \times V} \underbrace{\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u)}_{V \times V} \underbrace{\mathcal{G}^t(\mathbf{x}_u, \mathbf{y}_u)}_{V \times 1} + \mathcal{O}(\eta^2 \|\nabla_{\theta} \mathbf{z}(\mathbf{x}_u)\|_{\text{op}}^2)$$

$$\mathcal{A}^t(\mathbf{x}_o) = \nabla_{\mathbf{z}} \log \pi_{\theta^t}(\mathbf{x}_o) = I - \mathbf{1} \pi_{\theta^t}^{\top}(\mathbf{x}_o)$$

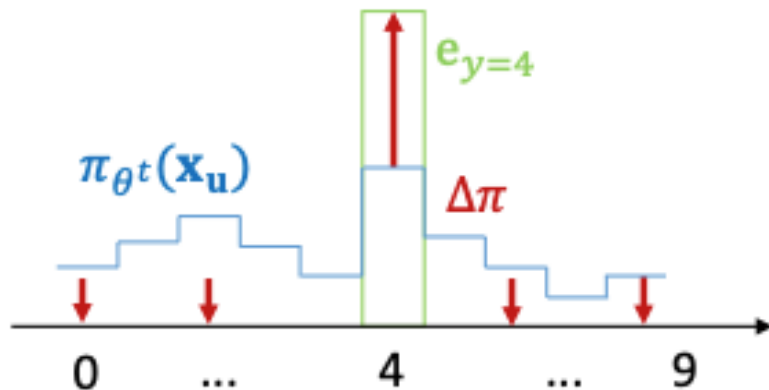
$$\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u) = (\nabla_{\theta} \mathbf{z}(\mathbf{x}_o)|_{\theta^t})(\nabla_{\theta} \mathbf{z}(\mathbf{x}_u)|_{\theta^t})^{\top}$$

$$\mathcal{G}^t(\mathbf{x}_u, \mathbf{y}_u) = \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{x}_u, \mathbf{y}_u)|_{\mathbf{z}^t}$$

During the training, the relative influence of learning x_u on all other different x_o is relatively stable.

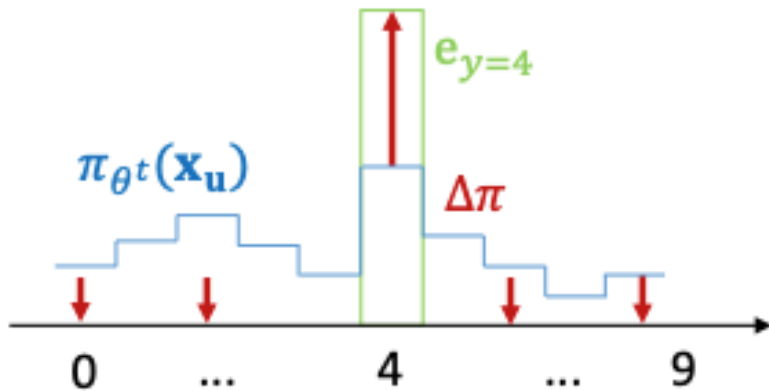
Learning Dynamic in action: MNIST

Learn $(\mathbf{x}_u = 4, y_u = 4)$ using SGD



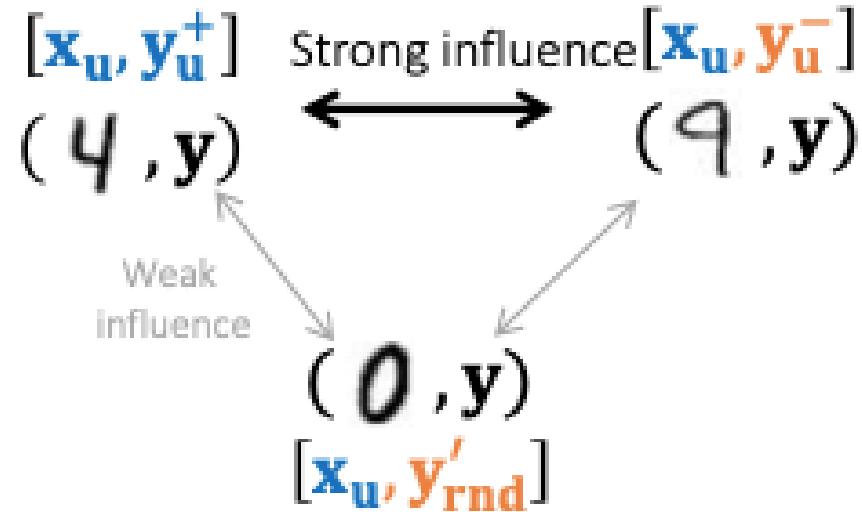
Learning Dynamic in action: MNIST

Learn $(\mathbf{x}_u = 4, y_u = 4)$ using SGD

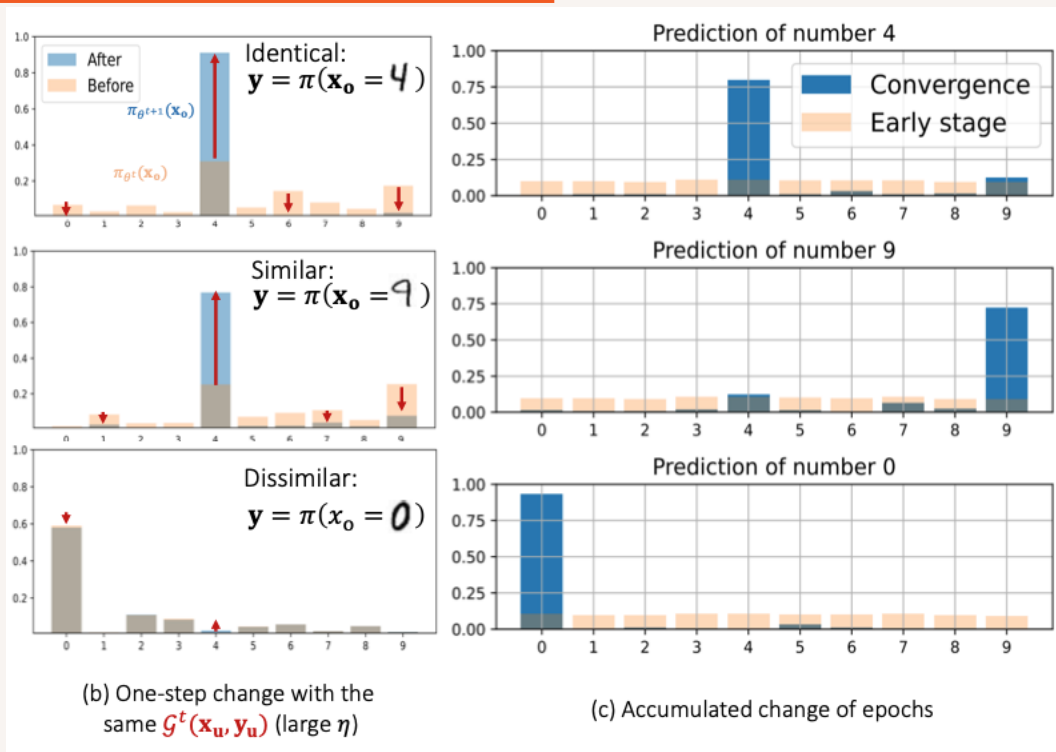


$$\mathcal{G}^t(\mathbf{x}_u, \mathbf{y}_u) = \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{x}_u, \mathbf{y}_u)|_{\mathbf{z}^t}$$

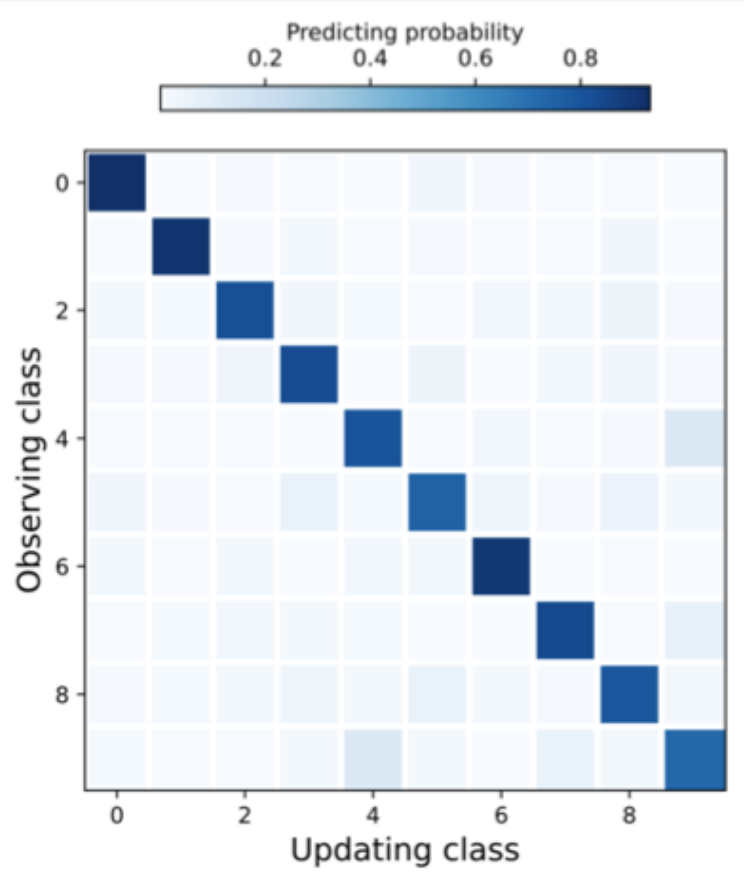
Learning Dynamic in action: MNIST



Learning Dynamic in action: MNIST



Learning Dynamic in action: MNIST



Learning Dynamic of LLM Finetuning:

$$\mathcal{L}_{\text{SFT}}(\mathbf{x}_u, \mathbf{y}_u^+) \triangleq - \sum_{l=1}^L \underbrace{\log \pi(y = y_l^+ \mid y_{<l}^+, \mathbf{x}_u)}_{\text{probability of the correct } l\text{-th token}} = - \sum_{l=1}^L \mathbf{e}_{y_l^+}^\top \log \pi(\mathbf{y} \mid \mathbf{x}_u, y_{<l}^+)$$

$$\underbrace{\pi(\mathbf{y} \mid \mathbf{x}) = \prod_l \pi(y_l \mid \mathbf{x}, y_{<l})}_{\text{sequential: token } l \text{ depends on all earlier tokens}}$$

teacher forcing: $y_{<l}$ is *given* $\rightarrow$
causal mask inside h_θ

$$\underbrace{\chi \triangleq [\mathbf{x}; \mathbf{y}]}_{\text{one input, columns now independent}}$$

$$\underbrace{z}_{V \times L} = h_\theta(\chi), \quad \pi(\mathbf{y} \mid \chi) = \text{Softmax_column}(z) \quad \Rightarrow$$

a multi-label problem
columns of z handled separately

Learning Dynamic of LLM Finetuning:

$$\underbrace{[\Delta \log \pi^t(\mathbf{y} \mid \boldsymbol{\chi}_o)]_m}_{V \times M} = - \sum_{l=1}^L \eta \underbrace{[\mathcal{A}^t(\boldsymbol{\chi}_o)]_m}_{V \times V \times M} \underbrace{[\mathcal{K}^t(\boldsymbol{\chi}_o, \boldsymbol{\chi}_u)]_{m,l}}_{V \times V \times M \times L} \underbrace{[\mathcal{G}^t(\boldsymbol{\chi}_u)]_l}_{V \times L} + \mathcal{O}(\eta^2)$$

What were we forced to do to come up to a similar formulation?

DPO Crash Course:: Umar Jamil

The RL setup

Agent: the cat

State: the position of the cat (x, y) in the grid

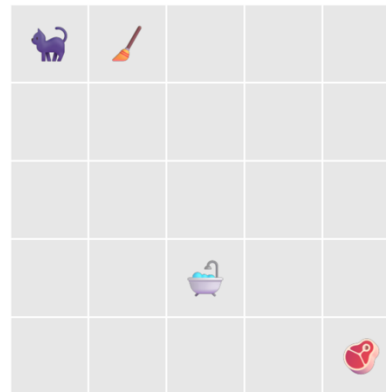
Action: at each position, the cat can move to one of the 4-directionally connected cells. If a move is invalid, the cell will not move and remain in the same position. Every time the cat makes a move, it results in a new state and a reward.

Reward model:

- A move to another empty cell results in a reward of 0.
- A move towards the broom, will result in a reward of -1.
- A move towards the bathtub will result in a reward of -10 and the cat fainting (episode over). The cat will be respawned at the initial position again.
- A move towards the meat will result in a reward of +100

Policy: a policy rules how the agent selects the action to perform given the state it is in:
 $a_t \sim \pi(\cdot | s_t)$

The goal in RL is to select a policy that maximizes the expected return when the agent acts according to it.



The RL setup: connection to language models

Agent: the language model itself

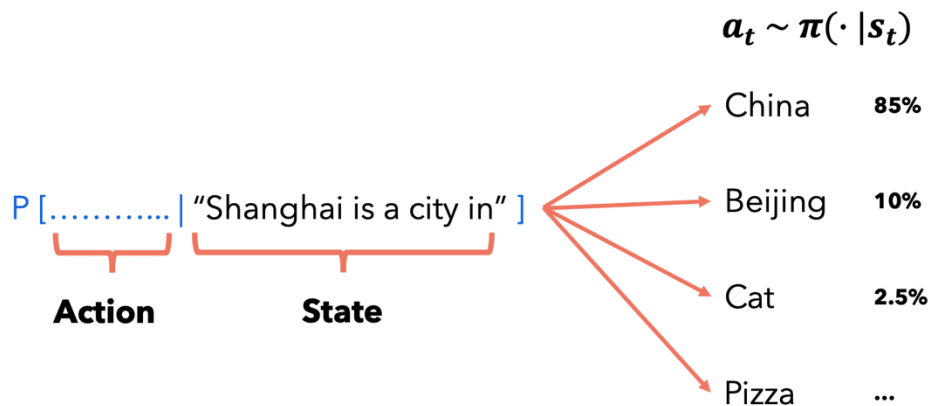
State: the prompt (input tokens)

Action: which token is selected as the next token

Reward model: the language model should be rewarded for generating “good responses” and should not receive any reward for generating “bad responses”.

Policy: In the case of language models, the policy is the language model itself! Because it models the probability of the action space given the current state of the agent: $a_t \sim \pi(\cdot | s_t)$

Let's look at how we can define the reward model for our language model



Reward model for language models

It is not easy to create a reward model for language models, because this would require us to create a dataset of prompts and responses and assign a universally accepted “reward” for each answer.

Question (Prompt)	Answer (Text generated by the language model)	Reward (0.0 ~ 1.0)
Where is Shanghai?	Shanghai is a city in China	???
Explain gravity like I'm 5	Gravity is what pulls things toward each other. It's why you stay on the ground and planets orbit the sun.	???
What is 2+2?	4	???

People are so good good at finding a common ground for agreement, but unfortunately, they're good at comparing.

The Bradley-Terry model

Now that we have a dataset of preferences, we need to find a way to convert the preferences into a score (reward). One way to do it is to model our preferences using the Bradley-Terry model.

$$P(y_w > y_l) = \frac{e^{r^*(x, y_w)}}{e^{r^*(x, y_w)} + e^{r^*(x, y_l)}}$$

Question/Prompt (x)	Good/winning answer (y_w)	Bad/losing answer (y_l)
Where is Shanghai?	Shanghai is a city in China	Shanghai does not exist
Explain gravity like I'm 5	Gravity is what pulls things toward each other. It's why you stay on the ground and planets orbit the sun.	Gravity is a famous restaurant
What is 2+2?	4	2+2 is a very complicated math problem...

Of course we want to maximize the probability that the preference model ranks our responses correctly. One way to do it is Maximum Likelihood Estimation. With MLE we can estimate the parameters of a reward model r^* such that the probability of ranking correctly the good and the bad answers is maximized. **Let's see how we can do it!**

The RLHF objective

Our goal is to solve the following constrained optimization: we want to find the policy that maximizes the reward obtained when using that policy and at the same time we want the policy to not behave too differently from the original unoptimized policy. This constraint is added to avoid what is known as “reward hacking”: the language model (the policy) may just choose sequences of tokens that achieve high reward but may be total gibberish.

$$J_{\text{RLHF}} = \max_{\pi_{\theta}} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(y|x)} \left[r_{\phi}(x, y) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta}(y|x) \parallel \pi_{\text{ref}}(y|x)] \right]$$



Maximize the reward



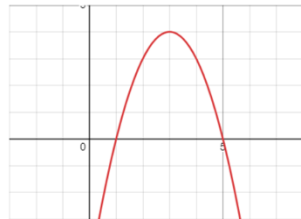
Constraint the model to be not so different from the original one

What does it mean to maximize an objective function?

The optimization problem

Maximizing a function means to find all the values of one or more variables such that the function is maximized. The same applies for minimization problems. For example, if we have a very simple function like the following: $f(x) = -(x - 3)^2 + 4$, to maximize it means to find the values of the variable x such that the function is maximized. One way to do it is to calculate the derivative of the function w.r.t to the variable x and set it to zero. In our case, the derivative is $f'(x) = -2x + 6$. If we set it to zero and solve for x , we find that $x^* = 3$ is the value that maximizes the function.

$$x^* = \operatorname{argmax}_x -(x - 3)^2 + 4$$



The simple parabola problem is an unconstrained optimization problem. In the case of RLHF we have a **constrained optimization** problem, which means that we want to maximize the reward but at the same time we want the KL divergence to be small.

$$J_{\text{RLHF}} = \max_{\pi_{\theta}} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(y|x)} \left[r_{\phi}(x, y) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta}(y|x) \parallel \pi_{\text{ref}}(y|x)] \right]$$

If you're thinking "why not just run gradient descent on this objective function"? Well, we can't, because the variable y is sampled from the language model itself using various strategies (greedy, beam search, top-k, and similar). **This sampling process is not differentiable. This is the reason we were forced to use RL algorithms like PPO.**

The optimal policy

The solution to the constrained optimization problem is the following. It is the equation (4) in the DPO paper.

$$\pi_r(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Where $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$. **Great! We have an exact solution to this constrained optimization problem, so now we're all set... right?**

Not really. Evaluating the $Z(x)$ term is not tractable computationally, because it means we would have to generate all possible answers y that can be generated by our language model for every given prompt x .

Suppose that somehow, magically, we have access to an optimal policy $\pi_r(y|x)$... how would the reward function $r(x, y)$ be for such an optimal policy?

$$\log \pi^*(y|x) = \log \left[\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \right] = \log \pi_{\text{ref}}(y|x) - \log Z(x) + \log \exp\left(\frac{1}{\beta} r(x, y)\right) = \log \pi_{\text{ref}}(y|x) - \log Z(x) + \frac{1}{\beta} r(x, y)$$

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

So, if we had access to an optimal policy, we could use it to obtain a reward function. **OK, but what can we do with it now?**

... remember the Bradley-Terry model?

$$P(y_w > y_l) = \frac{e^{r(x, y_w)}}{e^{r(x, y_w)} + e^{r(x, y_l)}} \quad \dots \text{ and as shown in previous slides...} \quad \frac{e^A}{e^A + e^B} = \dots = \frac{1}{1 + e^{B-A}} = \frac{1}{1 + e^{-(A-B)}} = \sigma(A - B)$$

What if we plugged the expression of the reward function into the expression of the Bradley-Terry model?

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

$$P(y_w > y_l) = \sigma(r(x, y_w) - r(x, y_l)) = \sigma \left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} + \beta \log Z(x) - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log Z(x) \right)$$

To learn a Bradley-Terry model that maximizes the probability of choosing y_w over y_l , we just need to maximize the expression above or minimize the negative expression below:

$$L_{DPO}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

So instead of optimizing the reward function, we are optimizing the optimal policy (which depends on the optimal reward function through the expression in the previous slide). Brilliant solution!

DPO Loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\mathbf{x}_u, \mathbf{y}_u^+, \mathbf{y}_u^-) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta^t}(\mathbf{y}_u^+ | \mathbf{x}_u^+)}{\pi_{\text{ref}}(\mathbf{y}_u^+ | \mathbf{x}_u^+)} - \beta \log \frac{\pi_{\theta^t}(\mathbf{y}_u^- | \mathbf{x}_u^-)}{\pi_{\text{ref}}(\mathbf{y}_u^- | \mathbf{x}_u^-)} \right) \right]$$

$$[\Delta \log \pi^t(\mathbf{y} | \mathbf{x}_o)]_m = - \sum_{l=1}^L \eta [\mathcal{A}^t(\mathbf{x}_o)]_m ([\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u^+)]_{m,l} [\mathcal{G}_{\text{DPO}^+}^t]_l - [\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u^-)]_{m,l} [\mathcal{G}_{\text{DPO}^-}^t]_l) + \mathcal{O}(\eta^2)$$

There is no assumption on where do + and – samples come from

The Squeezing Effect

$$[\Delta \log \pi^t(\mathbf{y} \mid \mathbf{x}_o)]_m = - \sum_{l=1}^L \eta [\mathcal{A}^t(\mathbf{x}_o)]_m ([\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u^+)]_{m,l} [\mathcal{G}_{\text{DPO}^+}^t]_l - [\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u^-)]_{m,l} [\mathcal{G}_{\text{DPO}^-}^t]_l) + \mathcal{O}(\eta^2)$$

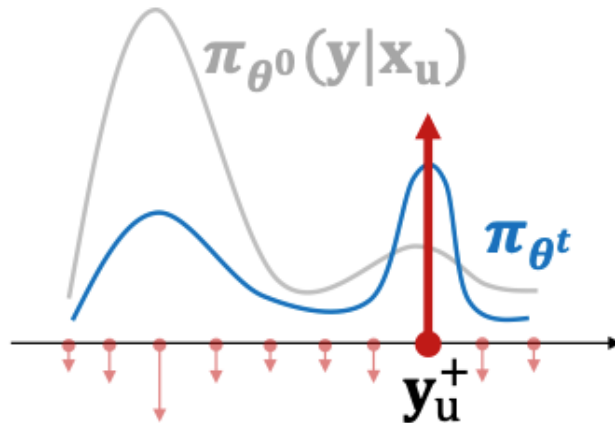
The Squeezing Effect

$$[\Delta \log \pi^t(\mathbf{y} \mid \mathbf{x}_o)]_m = - \sum_{l=1}^L \eta [\mathcal{A}^t(\mathbf{x}_o)]_m ([\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u^+)]_{m,l} [\mathcal{G}_{\text{DPO}^+}^t]_l - [\mathcal{K}^t(\mathbf{x}_o, \mathbf{x}_u^-)]_{m,l} [\mathcal{G}_{\text{DPO}^-}^t]_l) + \mathcal{O}(\eta^2)$$

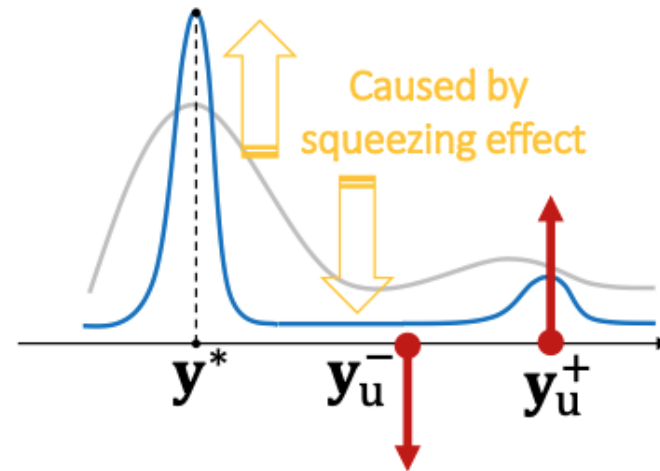
- *Guarantee:* the confidence of $\mathbf{y}_u^-$, i.e., $\pi_{\theta^{t+1}}(\mathbf{y}_u^-)$ will decrease.
- *Guarantee:* the decreased probability mass is largely “squeezed” into the output which was most confident before the update: if $\mathbf{y}^* = \operatorname{argmax}_{i \in [V] \setminus \{\mathbf{y}_u^-\}} \pi_{\theta^t}(\mathbf{y} = i)$, then $\pi_{\theta^{t+1}}(\mathbf{y} = \mathbf{y}^*)$ will increase.
- *Trend:* the rich get richer and the poor get poorer: generally, dimensions with high π_{θ^t} tend to increase, and those with low π_{θ^t} tend to decrease.
- *Trend:* peakier π_{θ^t} squeezes more. If the probability mass concentrates on few dimensions in π_{θ^t} , which is common for a pretrained model, all $\pi_{\theta^{t+1}}(\mathbf{y} \neq \mathbf{y}^*)$ decrease (only $\mathbf{y}^*$ is “rich”).
- *Trend:* smaller $\pi_{\theta^t}(\mathbf{y}_u^-)$ exacerbate the squeezing effect: if $\mathbf{y}_u^-$ is unlikely under π_{θ^t} , the probability mass of all other $\pi_{\theta^{t+1}}(\mathbf{y} \neq \mathbf{y}^*)$ will be more seriously decreased, and the $\pi_{\theta^{t+1}}(\mathbf{y} = \mathbf{y}^*)$ increases more.

The Squeezing Effect

- SFT



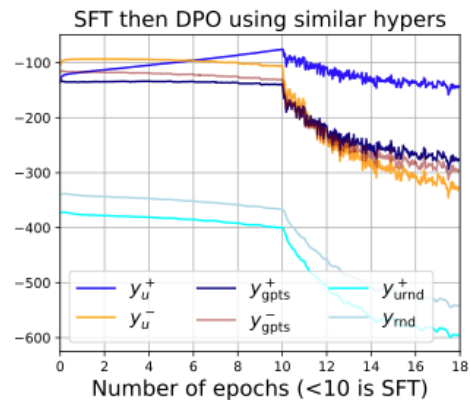
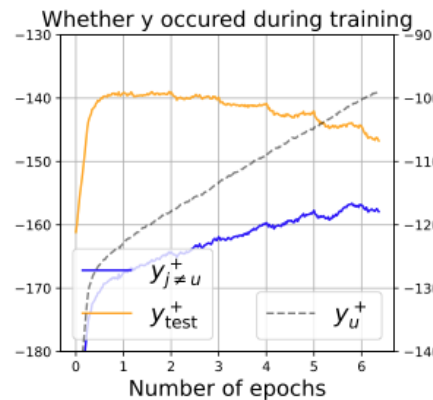
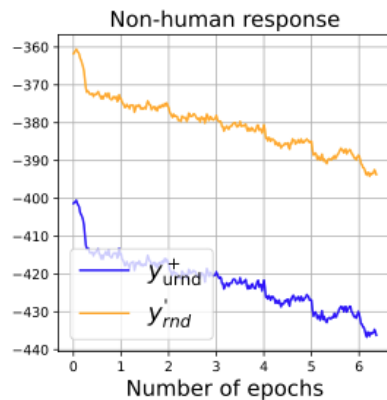
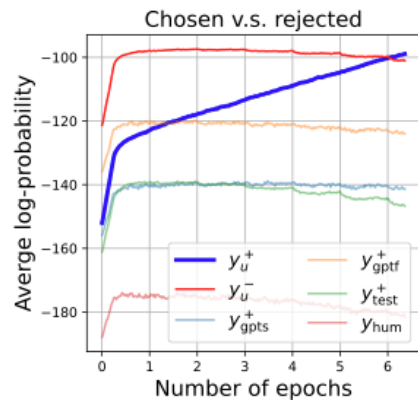
- Off-policy DPO, IPO



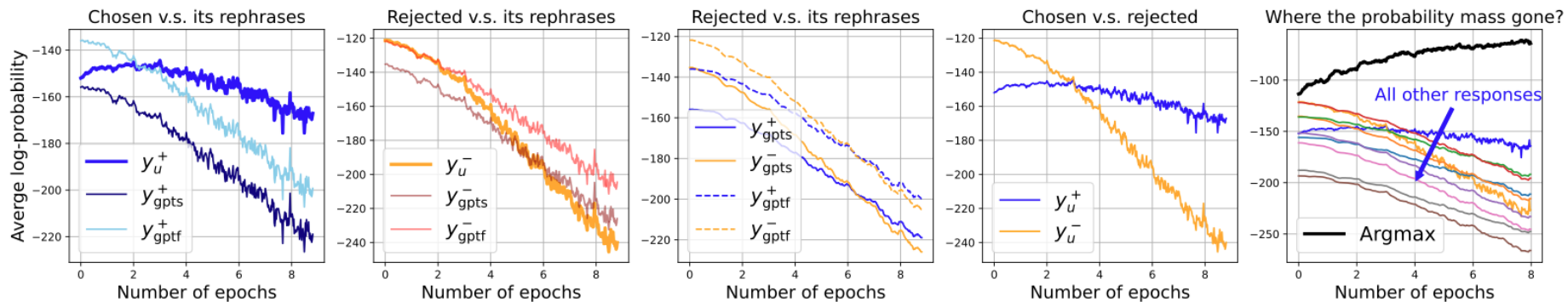
Experiment Setup

- 5k examples, Anthropic-HH + UltraFeedback; triples (x, y^+, y^-)
- SFT: (x, y^+) · DPO: (x, y^+, y^-)
- Pythia 410M–2.8B, Qwen1.5 0.5B/1.8B
- Probing set: 500 examples × several response types (rephrases, other-question answers, random sentences)
- Track $\log \pi(y \mid x)$ per type:: probing questions as the space is vast $\mathcal{Y} \in \mathbb{R}^{V \times L}$

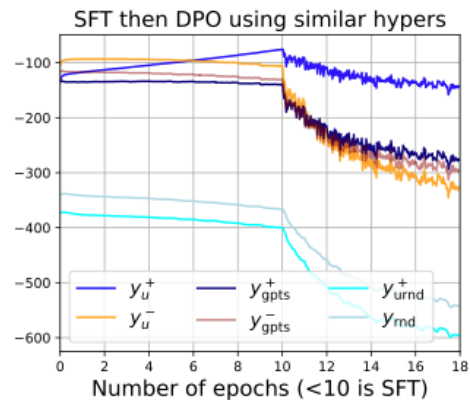
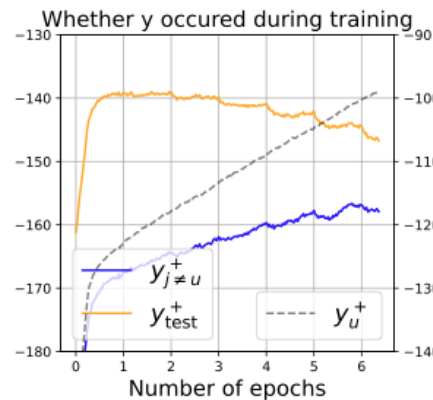
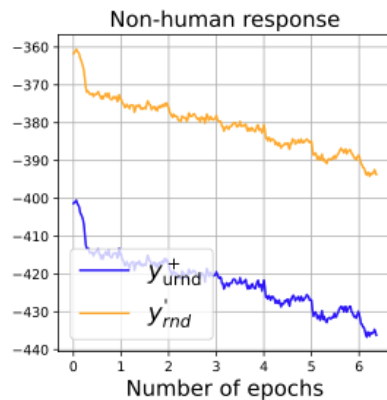
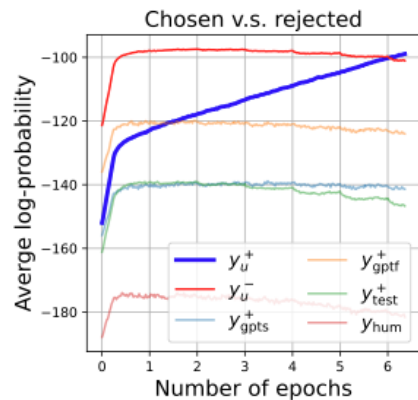
Discuss



Discuss

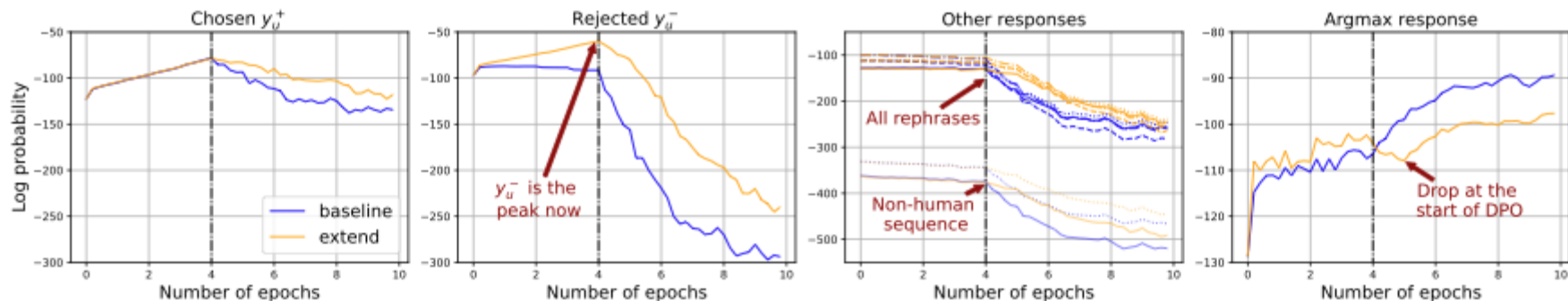


Discuss



Solution

We can first train the model on both $[x_u; y^+_u]$ and $[x_u; y^-_u]$ during the SFT stage (making the negative response more likely), and then run the usual DPO



Why Best Paper?

- Relatively simple analysis
- Very simple but exhaustive math
- Good demonstrations and practical suggestions for training regime

References

- <https://arxiv.org/abs/2407.10490>
- <https://huggingface.co/blog/karina-zadorozhny/guide-to-llm-post-training-algorithms>
- <https://arxiv.org/pdf/2206.12543>
- <https://arxiv.org/pdf/2305.18290>